UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Desenvolvimento de um Sistema Interativo para Monitoramento e Análise do Fluxo de Materiais Doados na Biblioteca Padre Jaime Diniz utilizando Python e Streamlit

Gabriel de Oliveira Barreto

Orientador: Prof. Dr. Luiz Affonso Henderson Guedes de Oliveira

Natal, RN, 7 de julho de 2026

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Desenvolvimento de um Sistema Interativo para Monitoramento e Análise do Fluxo de Materiais Doados na Biblioteca Padre Jaime Diniz utilizando Python e Streamlit

Gabriel de Oliveira Barreto

Orientador: Prof. Dr. Luiz Affonso Henderson Guedes de Oliveira

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 7 de julho de 2026

Desenvolvimento de um Sistema Interativo para Monitoramento e Análise do Fluxo de Materiais Doados na Biblioteca Padre Jaime Diniz utilizando Python e Streamlit

Gabriel de Oliveira Barreto

Monografia aprovada em 7 de julho de 2026, pela banca examinadora composta pelos seguintes membros:

Prof. Dr. Luiz Affonso Henderson Guedes de Oliveira (orientador) . . . . . .
DCA/UFRN

Eng. Ernane Ferreira Rocha Junior . . . . . DCA/UFRN

Dr. Everton Rodrigues Barbosa . . . . . EMUFRN

Aos meus .....

Agradecimentos

Ao meu orientador, o professor Luiz Affonso Henderson Guedes de Oliveira, sou grato pela orientação e pela compreensão.

Aos meus amigos que fiz durante a graduação, obrigado por me apoiarem e acreditarem em mim, dando-me forças para continuar minha jornada.

E, principalmente, aos meus pais, que fizeram de tudo para cuidar de mim e me dar a educação necessária para que eu pudesse chegar até aqui.

Resumo

O presente trabalho apresenta o desenvolvimento de um painel analítico interativo (dashboard) automatizado, voltado a otimizar o monitoramento do fluxo de processamento técnico de materiais da Biblioteca Setorial da Escola de Música Padre Jaime Diniz (BPJD/UFRN). A aplicação propõe-se a resolver as limitações do acompanhamento histórico baseado em planilhas estáticas do Google Sheets, formato tabular que dificultava a identificação imediata de gargalos operacionais, a distribuição interna de tarefas por profissional e o cálculo automatizado do acúmulo de pendências (*backlog*). Além disso, o sistema soluciona a lentidão na consolidação de dados administrativos, simplificando substancialmente a elaboração dos relatórios mensais de produtividade encaminhados à Biblioteca Central Zila Mamede (BCZM). Implementada integralmente no ecossistema Python com arquitetura orientada a objetos, a solução utiliza as bibliotecas Streamlit e Plotly no front-end para entregar uma interface reativa com gráficos dinâmicos, e as bibliotecas Gspread, Pandas e Datetime no back-end para a extração síncrona via API e manipulação dos dados em tempo real. Os testes com dados reais comprovaram a eficácia da ferramenta em isolar o passivo operacional acumulado do setor e prover filtros dinâmicos por data, tipo de material e operador, consolidando-se como um mecanismo prático de inteligência de dados para apoio à tomada de decisões gerenciais.

Palavras-chave: Visualização de Dados, Processamento Técnico, Streamlit, BPJD.

Abstract

This paper presents the development of an automated interactive analytical dashboard designed to optimize the monitoring of the material technical processing workflow at the Padre Jaime Diniz Music School Sectoral Library (BPJD/UFRN). The application aims to solve the limitations of tracking historical records through static Google Sheets spreadsheets, a tabular format that hindered the immediate identification of operational bottlenecks, task distribution among professionals, and the automated calculation of accumulated pending tasks (backlog). Furthermore, the system addresses the delays in consolidating administrative data, substantially simplifying the elaboration of monthly productivity reports submitted to the Zila Mamede Central Library (BCZM). Fully implemented within the Python ecosystem using an object-oriented architecture, the solution leverages Streamlit and Plotly on the front-end to deliver a reactive user interface with dynamic charts, and Gspread, Pandas, and Datetime on the back-end for synchronous API data extraction and real-time matrix manipulation. Tests conducted with real-world data proved the tool's efficacy in isolating the sector's accumulated operational liabilities and providing dynamic filters by date, material type, and operator, establishing itself as a practical business intelligence mechanism to support management decision-making.

Keywords: Data Visualization, Technical Processing, Streamlit, BPJD.

Sumário

Sumário i

Lista de Figuras iii

Lista de Tabelas v

1 Introdução 1

2 Fundamentação Teórica 3
- 2.1 Classes (Orientação a Objetos) . . . 3
- 2.2 Google Cloud Platform (GCP) . . . 3
- 2.3 Gspread . . . 3
- 2.4 Pandas . . . 4
- 2.5 Datetime . . . 4
- 2.6 Plotly . . . 4
- 2.7 Streamlit . . . 4

3 Metodologia / Implementação 5
- 3.1 Arquitetura Geral do Sistema . . . 5
- 3.2 Camada de Dados . . . 6
  - 3.2.1 Configuração do Google Cloud e Autenticação via Conta de Serviço 6
  - 3.2.2 Função: buscar_dados_brutos_da_planilha() . . . 7
  - 3.2.3 Classe: DadosBiblioteca . . . 7
- 3.3 Camada de Visualização . . . 10
  - 3.3.1 Classe: Graficos . . . 10
  - 3.3.2 Classes Especializadas . . . 12
- 3.4 Camada de Apresentação . . . 14
  - 3.4.1 Ponto de Entrada (app.py) . . . 15
  - 3.4.2 Página Inicial (inicio.py) . . . 16
  - 3.4.3 Painel Principal (estatistica.py) . . . 16
- 3.5 Interface e Especificação Funcional dos Módulos do Painel . . . 18
  - 3.5.1 Módulo: Visão Geral . . . 18
  - 3.5.2 Módulo: Doações . . . 18
  - 3.5.3 Módulo: Tombamento . . . 19
  - 3.5.4 Módulo: Catalogação . . . 20
  - 3.5.5 Módulo: Estante de Novidades . . . 20

3.5.6 Módulo: Preparo Final 21

4 Experimentos e Resultados **23**
4.1 Cenário Anterior à Implementação 23
4.2 Sistema Implementado 25

5 Conclusão **33**

Referências bibliográficas **34**

Lista de Figuras

3.1 Função para buscar os dados brutos da planilha do *Google Sheets*. 7
3.2 Método construtor da classe DadosBiblioteca. 8
3.3 Método que busca e faz o tratamento dos dados da classe DadosBiblioteca. 9
3.4 Método que retornar um DataFrame filtrado por um intervalo de datas da classe DadosBiblioteca. 9
3.5 Diagrama UML de Classes. 10
3.6 Método construtor da classe Graficos. 10
3.7 Trecho do algoritomo de backlog empregado no método renderizar_metricas() da classe GraficosTombamento. 14
3.8 *Script* completo do arquivo app.py. 15
3.9 Trecho do código de validação de datas. 16
3.10 Trecho do código do mapeamento dinâmico das classe através de um dicionário. 17
3.11 Diagrama UML de sequência. 17

4.1 Estrutura tabular da planilha de fluxo de recebimento, com registros inseridos manualmente por aba. 24
4.2 Aba "Estatística"da planilha original, com filtros de data editados manualmente e totais calculados pela função `SOMA`. 24
4.3 Tela inicial do sistema. 25
4.4 Barra lateral com navegação de páginas e seletores de filtros de data e categoria. 26
4.5 Painel do Módulo de Visão Geral. 27
4.6 Gráfico de funil das etapas do fluxo de processamento. 27
4.7 Tabela de registros. 28
4.8 Mensagem de erro exibida na validação do filtro de datas. 28
4.9 Gráfico de Barras da quantidade de doações recebidas por doador. 29
4.10 Gráfico de Pizza de quantidade de tombamentos por tipo de material. . . . 29
4.11 Gráfico Sunburst de catalogação por responsável e tipo de material. . . . 30
4.12 Painel de indicadores da categoria Preparo Final. 30
4.13 Gráfico de Barras Agrupadas da categoria Preparo Final. 31

Lista de Tabelas

Capítulo 1

Introdução

Bibliotecas universitárias desempenham um papel central no apoio às atividades de ensino, pesquisa e extensão. Para que possam cumprir essa função de forma eficiente, é necessário que os processos internos de tratamento e organização do acervo sejam realizados de maneira sistemática e documentada. O processamento técnico de um material bibliográfico compreende um conjunto sequencial de etapas que vai desde o recebimento do material informacional até a sua disponibilização nas estantes para uso dos usuários. Esse fluxo envolve atividades como a seleção e avaliação técnica do material recebido, o tombamento patrimonial, a catalogação descritiva e temática, a indexação e a preparação física final — que inclui a impressão e colagem de etiquetas e a inserção de papeletas. Cada uma dessas etapas é responsabilidade de um profissional específico, e o controle sobre esse fluxo é fundamental para garantir a rastreabilidade e a eficiência operacional da biblioteca.

Na Biblioteca Setorial da Escola de Música Padre Jaime Diniz (BPJD), integrada ao Sistema de Bibliotecas (SISBI), pertencente à Universidade Federal do Rio Grande do Norte (UFRN), esse fluxo operacional lida diariamente com um volume expressivo e diversificado de itens, tais como livros, partituras, CDs e songbooks. Com o objetivo de controlar e monitorar essas atividades inerentes aos materiais incorporados, a biblioteca instituiu em meados de 2019 uma planilha de fluxo de recebimento de doações. Em 2024, esse instrumento foi aperfeiçoada e estruturado em conformidade com o fluxograma oficial de processamento da instituição, mapeando detalhadamente as seis macroatividades sequenciais que compõem o ciclo técnico da unidade.

A primeira delas é a "Doação", que concentra o recebimento frequente de materiais na biblioteca, registrando o título da obra, tipo, doador, data de chegada, o envio da nota de agradecimento e o número do processo gerado na mesa virtual do Registro no Sistema Integrado de Patrimônio, Administração e Contratos (SIPAC), fase em que também ocorre a seletiva dos itens que serão incorporados ao acervo. Em seguida, ocorre o "Tombamento", etapa em que os materiais são patrimonializados como bens permanentes da Universidade por meio do SIPAC da UFRN, onde são tombados com o número de patrimônio individual, o número do termo de doação, a data do cadastro e a identificação do operador responsável. Na sequência, a "Catalogação"realiza a descrição detalhada e temática dos itens para viabilizar a recuperação das informações no catálogo on-line, coletando o número do registro catalográfico gerado no SIGAA, o número de chamada da

obra, o controle de revisão final da catalogação, a data e o operador responsável. Após essa fase, o fluxo direciona-se para a "Estante de Novidades", etapa voltada à divulgação de novas aquisições que envolve o registro de referências bibliográficas, a digitalização de capas e sumários para inclusão nas listagens digitais e o upload de arquivos em formato PDF diretamente no Sistema Integrado de Gestão de Atividades Acadêmicas (SIGAA), controlando as respectivas datas de execução e bolsistas responsáveis. Concluídos esses registros, o "Preparo Final"executa as intervenções físicas necessárias para a disponibilização nas estantes, monitorando as datas e responsáveis pela carimbagem de identificação, impressão e colagem de etiquetas de localização (dorso e código de barras) e inserção de papeletas de empréstimo. Por fim, a "Revisão Final"antecede o encaminhamento dos materiais para o acervo físico, sendo realizada pelo bibliotecário coordenador, que atua como validador do ciclo ao verificar se todos os procedimentos anteriores foram executados de forma apropriada, evitando a inserção de itens com inconsistências ou etiquetas incorretas no acervo.

Embora essa abordagem garanta a persistência das informações coletadas desde 2019, a visualização isolada e puramente tabular desses dados no Google Sheets dificulta a identificação imediata de gargalos operacionais entre as seis etapas, a análise de produtividade por profissional e o cálculo automatizado do acúmulo de pendências (*backlog*) histórico. Como consequência, a consolidação manual de informações administrativas tornava complexa e morosa a elaboração dos relatórios mensais de produtividade que devem ser obrigatoriamente encaminhados à Biblioteca Central Zila Mamede (BCZM). Diante desse cenário, justifica-se o desenvolvimento de uma ferramenta computacional dedicada à consolidação e à inteligência de dados, capaz de transformar registros brutos em indicadores visuais dinâmicos para apoiar a gestão da biblioteca. A arquitetura da solução proposta divide-se em um *front-end* reativo construído com o *framework Streamlit* e suporte a gráficos dinâmicos e interativos via *Plotly*, e um *back-end* integrado em tempo real à API do *Google Sheets* por meio da biblioteca *Gspread*, utilizando *Pandas e Datetime* para a limpeza e manipulação das matrizes de dados. Essa abordagem de engenharia de software fundamenta-se na necessidade de criar uma solução ágil, responsiva e de baixo custo de manutenção, fornecendo à coordenação uma visualização centralizada da produtividade de bolsistas e funcionários.

O objetivo geral deste trabalho é desenvolver um painel de análise de dados em Python para monitorar o fluxo de recebimento e processamento técnico de materiais da BPJD/UFRN. A ferramenta permite filtrar as informações por data, tipo de material e operador responsável. Para alcançar esse escopo, definem-se como objetivos específicos: construir um *pipeline* automatizado de extração de dados via API do *Google Sheets*; implementar algoritmos para o cálculo de *backlog* temporal independente; estruturar a aplicação sob o paradigma de orientação a objetos, garantindo a modularidade do código; e disponibilizar uma interface intuitiva, organizada por módulos analíticos que reflitam fielmente as etapas reais da instituição.

Capítulo 2

Fundamentação Teórica

Para criar uma aplicação que mostre o fluxo da biblioteca em tempo real, foi preciso unir conceitos de organização de código com ferramentas modernas de Ciência de Dados. Este capítulo explica as tecnologias e os conceitos que servem de base para o desenvolvimento do sistema da Biblioteca Padre Jaime Diniz (BPJD).

2.1 Classes (Orientação a Objetos)

O uso de classes permite estruturar o código sob o paradigma da Orientação a Objetos (POO). Elas são utilizadas para encapsular a lógica visual e os métodos de filtragem em blocos reutilizáveis. Através de conceitos como herança e polimorfismo, garante-se que novos painéis ou regras de negócio possam ser integrados ao sistema sem a necessidade de reescrever a estrutura principal do software.

2.2 Google Cloud Platform (GCP)

A plataforma de nuvem do Google é utilizada como infraestrutura para o gerenciamento seguro de acessos. Por meio da configuração de uma Conta de Serviço (Service Account), o sistema obtém uma identidade digital criptografada (via chave privada RSA). Isso permite que a aplicação se autentique perante os servidores do Google e acesse os dados necessários de maneira 100% automatizada, eliminando a necessidade de telas de login interativas.

2.3 Gspread

O Gspread é uma biblioteca Python construída especificamente para interagir com a API do Google Sheets. Ela atua como uma camada de abstração que simplifica requisições web complexas em comandos simples de código. No projeto, ela é a ferramenta responsável por se conectar de forma programática à planilha de controle da biblioteca, localizando os arquivos pelo nome e extraindo as linhas de registros em tempo de execução.

2.4 Pandas

O Pandas é a principal biblioteca de manipulação e análise de dados em Python. Ela introduz a estrutura de DataFrames, que organiza os dados brutos extraídos para o processamento. O Pandas é utilizado para realizar a limpeza de textos, ordenação, cruzamento de colunas e os agrupamentos numéricos necessários para gerar as métricas de volume de trabalho.

2.5 Datetime

A Datetime é uma biblioteca nativa do Python voltada para o processamento de datas e horários. No sistema, ela fornece o suporte para lidar com séries temporais. É a ferramenta utilizada para validar os intervalos escolhidos pelo usuário na interface, converter textos em instâncias temporais manipuláveis e realizar a varredura histórica necessária para o cálculo exato do acúmulo de tarefas pendentes (*backlog*).

2.6 Plotly

O Plotly é o motor gráfico utilizado para a construção das visualizações de dados. Ao contrário de ferramentas estáticas, ele gera gráficos dinâmicos e interativos baseados em ambiente web. A biblioteca foi aplicada para renderizar as visualizações de gráficos de barras, linhas, funil, pizza e a estrutura multinível Sunburst, fornecendo ao usuário recursos essenciais de exploração como zoom, isolamento de legendas e balões de informação (*hover*) detalhados ao passar o mouse.

2.7 Streamlit

O Streamlit é um framework em Python projetado para a criação rápida de aplicações web focadas em Ciência de Dados. Ele elimina a necessidade de codificar HTML, CSS ou JavaScript nativos, transformando scripts Python em páginas web interativas de forma declarativa. O principal recurso computacional empregado no sistema é o *st.session_state*, que gerencia o estado global da aplicação. Como o Streamlit reinicia o script a cada clique do usuário, esse mecanismo armazena em memória os filtros ativos e os dados já carregados, evitando requisições repetidas à API e garantindo uma navegação rápida entre as páginas.

Capítulo 3

Metodologia / Implementação

Este capítulo descreve a implementação técnica completa do sistema de monitoramento e visualização de dados da Biblioteca Padre Jaime Diniz (BPJD). São documentados todos os módulos, classes, métodos e os principais trechos de código que compõem a solução, bem como a configuração da infraestrutura de acesso à API do Google Sheets.

A ideia do sistema foi concebida no ambiente de trabalho da BPJD, com a necessidade de coletar e analisar os dados persistidos na planilha de fluxo de recebimento dos materiais doados à biblioteca, seja para a geração dos relatórios mensais enviados a BCZM, como para gestão interna de trabalho dos bolsistas e funcionários da biblioteca. A planilha contém as informações de cada etapa do processo: registro e seleção do material doado, tombamento patrimonial no SIPAC, catalogação bibliográfica no SIGAA, digitalização de capas e sumários com posterior upload no sistema, e preparo físico final, que inclui a impressão e colagem de etiquetas e a inserção de papeletas. Cada etapa envolve um responsável específico e gera dados que precisam ser rastreados e monitorados, o que apenas visualização em forma de tabelas dificultava as análises.

A abordagem seguida foi essencialmente iterativa, caracterizada por ciclos curtos de análise, implementação e refinamento conduzidos em contato contínuo com o usuário final da ferramenta. O desenvolvimento foi feito integralmente em Python, com o paradigma da Programação Orientada a Objetos. Com a utilização das bibliotecas Gspread, Pandas e Datetime, para extração, manipulação e análise dos dados. E as bibliotecas Streamlit e Plotly, para a criação das páginas web e visualização dos gráficos interativos.

3.1 Arquitetura Geral do Sistema

O sistema é composto por três camadas funcionais: a camada de dados, responsável pela integração com o Google Sheets e pelo tratamento dos registros brutos; a camada de visualização, que encapsula a lógica de geração de gráficos e o cálculo de indicadores; e a camada de apresentação, implementada com Streamlit, que gerencia a interface web, o estado da sessão e o roteamento entre páginas.

A separação entre essas camadas foi implementada por meio de módulos Python independentes, organizados no diretório src/. O ponto de entrada da aplicação é o arquivo app.py, que inicializa o estado global da sessão, instancia o gerenciador de dados e registra as páginas de navegação do Streamlit. A estrutura de diretórios do projeto é apresentada

a seguir:

```
Estatisticas_BPJD/
├── app.py                          # Ponto de entrada e estado global
├── credenciais.json                # Credenciais da API do Google
└── src/
    ├── gerenciador_dados.py        # Camada de dados
    ├── graficos.py                 # Classe base dos visualizadores
    ├── screens/
    │   ├── inicio.py               # Página de boas-vindas
    │   └── estatisticas.py         # Painel principal
    └── visualizers/
        ├── __init__.py
        ├── visao_geral.py
        ├── doacoes.py
        ├── tombamento.py
        ├── catalogacao.py
        ├── estante_novidade.py
        └── preparo_final.py
```

3.2 Camada de Dados

A camada de dados é implementada pela classe DadosBiblioteca, no módulo gerenciador_dados.py. Essa classe é responsável por autenticar na API do Google Sheets, recuperar os registros brutos da planilha e transformá-los em um DataFrame Pandas estruturado e pronto para análise. Antes de detalhar o módulo, é necessário descrever a configuração de infraestrutura no Google Cloud que viabiliza o acesso programático à planilha.

3.2.1 Configuração do Google Cloud e Autenticação via Conta de Serviço

O acesso à API do Google Sheets exige a criação de um projeto no Google Cloud Console e a configuração de uma conta de serviço com permissões específicas. Esse processo é realizado uma única vez e resulta em um arquivo de credenciais JSON que o sistema utiliza para se autenticar de forma programática, sem intervenção do usuário. As etapas de configuração são as seguintes:

1. Criação do projeto no Google Cloud Console: acessar console.cloud.google.com, criar um novo projeto e atribuir um nome identificador.
2. Ativação das APIs necessárias: no menu "APIs e Serviços", ativar a Google Sheets API e a Google Drive API. A ativação da Google Drive API é necessária porque a biblioteca Gspread utiliza internamente a Drive API para localizar a planilha pelo nome antes de acessá-la pela Sheets API.

3. Criação da conta de serviço: em "APIs e Serviços → Credenciais", criar uma conta de serviço com papel de leitor/editor, que representa a aplicação perante a API do Google, sem exigir login interativo de um usuário real.
4. Download do arquivo de credenciais: após criar a conta de serviço, gerar uma chave no formato JSON e fazer o download.
5. Compartilhamento da planilha com a conta de serviço: no Google Sheets, compartilhar a planilha de controle da BPJD com o endereço de e-mail da conta de serviço, concedendo permissão de leitura. Esse passo é indispensável, pois sem o compartilhamento explícito, a API retorna erro de acesso negado mesmo com credenciais válidas.

Com essa configuração estabelecida, a biblioteca Gspread consegue utilizar o arquivo credenciais.json para obter um token de acesso OAuth 2.0 automaticamente a cada sessão, sem necessidade de autenticação manual.

3.2.2 Função: buscar_dados_brutos_da_planilha()

Função de nível de módulo responsável pela autenticação na API do Google Sheets e pela leitura bruta da planilha. Recebe o caminho do arquivo de credenciais, o nome da pasta de trabalho e o nome da aba. Retorna uma lista de listas com todos os valores da aba. É decorada com @st.cache_data(ttl=120), de forma que a chamada à API seja reaproveitada por até 120 segundos entre interações do usuário.

```python
@st.cache_data(ttl=120)
def buscar_dados_brutos_da_planilha(creds_file, spreadsheet_name, worksheet_name):
    """Busca os dados brutos da planilha Google Sheets e armazena o resultado em cache.

    Esta função autentica usando credenciais de serviço, abre a planilha e
    retorna todas as células como uma lista de listas. O resultado é cacheado
    por 120 segundos para reduzir chamadas repetidas à API.

    Args:
        creds_file: Caminho para o arquivo JSON de credenciais do serviço.
        spreadsheet_name: Nome da planilha no Google Sheets.
        worksheet_name: Nome da aba dentro da planilha.

    Returns:
        Lista de listas com os valores de todas as células da aba selecionada.
    """
    scopes = ["https://spreadsheets.google.com/feeds", "https://www.googleapis.com/auth/drive"]
    creds = ServiceAccountCredentials.from_json_keyfile_name(creds_file, scopes) # type: ignore
    client = gspread.authorize(creds) # type: ignore

    planilha = client.open(title=spreadsheet_name).worksheet(worksheet_name)
    return planilha.get_all_values()
```

Figura 3.1: Função para buscar os dados brutos da planilha do *Google Sheets*.

3.2.3 Classe: DadosBiblioteca

Gerencia o ciclo de vida completo dos dados da biblioteca: autenticação, leitura, limpeza, normalização e filtragem. É instanciada uma única vez e armazenada em *st.session_state*

para evitar reconexões desnecessárias. Seu construtor armazena os parâmetros de conexão e inicializa o atributo df como None. A conexão efetiva com a API ocorre apenas na chamada explícita ao método buscar_dados().

```python
def __init__(self, credentials_file, spreadsheet_name, worksheet_name):
    """Inicializa a instância da classe DadosBiblioteca.

    Constrói o caminho para o arquivo de credenciais JSON, armazena os nomes
    da planilha e da aba, e inicializa o DataFrame como None até que os dados
    sejam carregados através do método buscar_dados().

    Args:
        credentials_file (str): Nome do arquivo JSON de credenciais do serviço,
            localizado em relação ao diretório 'config' do projeto.
        spreadsheet_name (str): Nome da planilha no Google Sheets a ser aberta.
        worksheet_name (str): Nome da aba (worksheet) dentro da planilha.
    """
    raiz_do_projeto = Path(__file__).resolve().parent.parent
    self.creds_path = raiz_do_projeto / "config" / credentials_file
    self.spreadsheet_name = spreadsheet_name
    self.worksheet_name = worksheet_name
    self.df = None
```

Figura 3.2: Método construtor da classe DadosBiblioteca.

buscar_dados()

Invoca buscar_dados_brutos_da_planilha() para obter os registros brutos e executa o pipeline completo de limpeza e normalização, resultando em um DataFrame Pandas estruturado armazenado em self.df. As transformações aplicadas são:

- Remoção de linhas de cabeçalho intermediárias inseridas manualmente na planilha, identificadas por critérios de conteúdo das células-chave.
- Mapeamento e filtragem das colunas de interesse.
- Mapeamento de campos booleanos: as colunas incorporacao_acervo e estante_novidades, registradas na planilha como "Sim"/"Não", são convertidas para True/False por dicionário de substituição.
- Conversão de colunas de data do formato string DD/MM/AAAA para objetos Python date via pd.to_datetime() com errors="coerce", transformando entradas inválidas ou ausentes em NaT.
- Remoção das linhas vazias que não contém data de registro na planilha.

```python
class DadosBiblioteca:
    def buscar_dados(self):
        dados = buscar_dados_brutos_da_planilha(self.creds_path, self.spreadsheet_name, self.worksheet_name)

        df_bruto = pd.DataFrame(dados)
        df_limpo = df_bruto.iloc[6:].copy()
        df_limpo.columns = df_bruto.iloc[6]
        df_limpo = df_limpo.iloc[1:]
        df_limpo = df_limpo.loc[:, df_limpo.columns.notna() & (df_limpo.columns != '')]

        df_filtrado = df_limpo.iloc[:, [c for c in self.MAPEAMENTO_COLUNAS.keys() if c < len(df_limpo.columns)]].copy()
        df_filtrado.columns = [self.MAPEAMENTO_COLUNAS[c] for c in self.MAPEAMENTO_COLUNAS.keys() if c < len(df_limpo.columns)]

        df_filtrado = df_filtrado.replace('', None)
        colunas_booleanas = ['incorporacao_acervo', 'estante_novidades']

        for coluna in colunas_booleanas:
            if coluna in df_filtrado.columns:
                df_filtrado[coluna] = df_filtrado[coluna].map(
                    {'Sim': True, 'Não': False, True: True, False: False}
                ).fillna(False)

        date_cols = [col for col in df_filtrado.columns if 'data' in col.lower()]
        for col in date_cols:
            df_filtrado[col] = pd.to_datetime(df_filtrado[col], format='%d/%m/%Y', errors='coerce').dt.date

        self.df = df_filtrado.dropna(subset=['data_registro'])
        return self.df
```

Figura 3.3: Método que busca e faz o tratamento dos dados da classe DadosBiblioteca.

filtrar_por_data(data_inicial, data_final, coluna_data)

Recebe um intervalo de datas e o nome de uma coluna de data e retorna uma cópia do DataFrame contendo apenas os registros cujo valor nessa coluna esteja dentro do intervalo [data_inicial, data_final]. É o único ponto de filtragem temporal do sistema, chamado pelos visualizadores via o método aplicar_filtro_por_coluna() da classe base Graficos.

```python
def filtrar_por_data(self, data_inicial, data_final, coluna_data='data_doacao'):
    """Retorna uma cópia do DataFrame filtrado por intervalo de datas.

    Filtra o DataFrame pelo intervalo de datas especificado em uma coluna
    indicada, retornando apenas os registros cujas datas estão dentro do
    intervalo fornecido (inclusivo).

    Args:
        data_inicial (date): Data inicial do intervalo de filtro (inclusiva).
        data_final (date): Data final do intervalo de filtro (inclusiva).
        coluna_data (str): Nome da coluna de data a ser utilizada para
            filtro. Padrão é 'data_doacao'.

    Returns:
        pandas.DataFrame: Cópia do DataFrame contendo apenas os registros
            cujas datas na coluna especificada estão dentro do intervalo
            de datas fornecido.
    """
    filtro = (self.df[coluna_data] >= data_inicial) & (self.df[coluna_data] <= data_final)
    return self.df.loc[filtro].copy() # type: ignore
```

Figura 3.4: Método que retornar um DataFrame filtrado por um intervalo de datas da classe DadosBiblioteca.

3.3 Camada de Visualização

3.3.1 Classe: Graficos

O módulo graficos.py contém a classe Graficos, que é a base de toda a camada de visualização. Ela encapsula os atributos compartilhados, o mecanismo de filtragem e todos os métodos de geração de gráficos. As seis classes especializadas herdam dessa base e sobrescrevem apenas os métodos renderizar_metricas() e renderizar_graficos(). O seu contrutor recebe o gerenciador de dados e o intervalo de datas do painel. Inicializa self.df como *None*; cada subclasse é responsável por chamar aplicar_filtro_por_coluna() antes de renderizar seus gráficos.

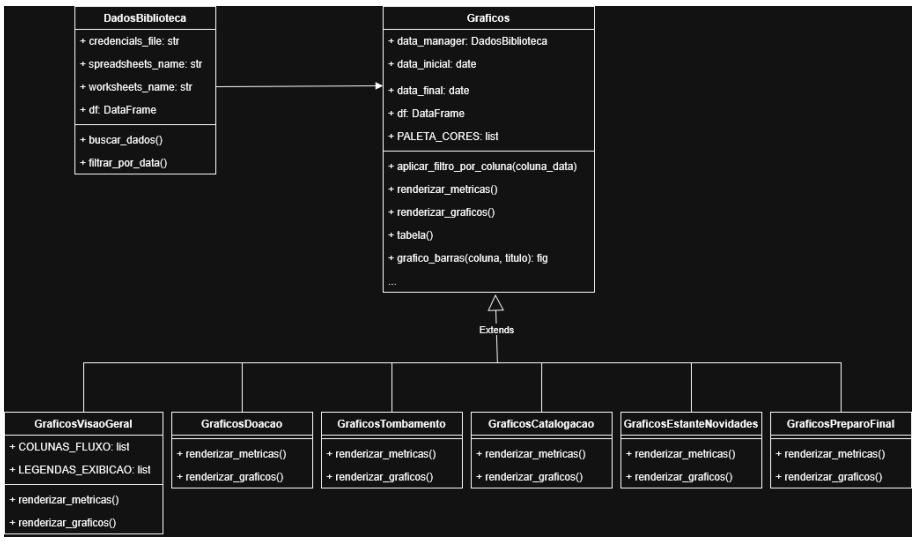


Figura 3.5: Diagrama UML de Classes.

```python
def __init__(self, data_manager, data_inicial, data_final):
    """Inicializa o objeto Graficos com dados e período de análise.

    Args:
        data_manager: Objeto responsável por acessar e filtrar os dados.
        data_inicial: Data inicial do intervalo de análise.
        data_final: Data final do intervalo de análise.

    O dataframe filtrado é armazenado em self.df após a aplicação do
    filtro por coluna de data.
    """
    self.data_manager = data_manager
    self.data_inicial = data_inicial
    self.data_final = data_final
    self.df = None
```

Figura 3.6: Método construtor da classe Graficos.

aplicar_filtro_por_coluna(coluna_data)

Delega ao método filtrar_por_data() do data_manager para filtrar self.df pelo intervalo de datas da instância usando a coluna especificada. É o mecanismo central de isolamento de contexto entre visualizadores, pois cada subclasse pode chamar esse método múltiplas vezes com colunas distintas para análises independentes.

grafico_barras(coluna, titulo)

Gera um gráfico de barras verticais agrupando os registros pela coluna informada. Calcula o total de cada categoria e constrói um hover customizado com a composição interna por tipo_material, exibindo a quebra de linha detalhada ao passar o cursor sobre cada barra. As barras são ordenadas do maior para o menor valor. Retorna *None* se self.df estiver vazio.

grafico_barras_horizontais(coluna, titulo)

Variante horizontal do gráfico de barras. Agrupa por coluna, ordena em ordem crescente e renderiza com *px.bar(orientation="h")*. Preferido quando os rótulos das categorias são longos, evitando sobreposição no eixo X.

grafico_barras_agrupadas(dicionario_etapas, coluna_categoria, titulo)

Gera um gráfico de barras agrupadas para comparar múltiplas séries sobre as mesmas categorias. O parâmetro dicionario_etapas mapeia o nome de cada série para sua definição de filtro, que pode ser:

- Uma string com o nome de uma coluna de data: aplica filtro temporal independente sobre essa coluna, sem depender do filtro principal da instância
- Uma tupla (coluna_booleana, valor): filtra os registros do período padrão onde a coluna booleana possui o valor especificado.

Para cada entrada do dicionário, uma cópia independente do DataFrame completo é processada, permitindo séries com origens de filtro completamente distintas no mesmo gráfico.

grafico_barras_cruzadas(coluna_x, coluna_cor, titulo, label_x, label_cor)

Gera um gráfico de barras agrupadas cruzando duas colunas categóricas do self.df já filtrado. Agrupa por (coluna_x, coluna_cor) e renderiza com *px.bar(barmode="group", color=coluna_cor)*. Utilizado para análises de dois eixos simultâneos, como tombamentos por responsável e tipo de material.

grafico_linhas(colunas_datas, nomes_legendas, titulo)

Gera um gráfico de linhas com múltiplas séries temporais. Para cada coluna de data informada, conta os registros por mês com *dt.to_period("M")* e consolida todas as séries em um único DataFrame antes de renderizar. Adequado para evidenciar tendências e comparar o volume mensal entre as etapas do fluxo.

grafico_pizza(coluna, title)

Gera um gráfico de pizza em formato de rosquinha (hole=0.4). Categorias com total menor ou igual a 10 são agrupadas automaticamente em "Outros", com hover customizado que detalha a composição interna do grupo. O agrupamento evita poluição visual em distribuições com muitas categorias minoritárias

grafico_sunburst(path, values, title)

Gera um gráfico sunburst hierárquico com *px.sunburst()*. O parâmetro *path* define a sequência de colunas que compõem a hierarquia de camadas (ex.: ["responsavel_tombamento", "tipo_material"]). Os valores de cada nó são calculados por *groupby()* sobre as colunas do *path*, contando os registros.

grafico_funil(etapas_colunas, nomes_etapas, titulo)

Gera um gráfico de funil que representa a eficiência de conversão do fluxo. Para cada coluna de data informada em etapas_colunas, conta os registros com valor não nulo (.notna().sum()), representando os materiais que completaram aquela etapa.

renderizar_graficos() e renderizar_metricas()

Métodos base que exibem mensagens informativas padrão e são obrigatoriamente sobrescritos pelas subclasses especializadas. Aplicam o padrão *Template Method*: a estrutura de chamada é definida na classe base e o conteúdo é implementado nas subclasses.

tabela()

Exibe um checkbox opcional "Exibir Tabela de Dados Filtrados". Quando marcado, renderiza self.df em formato tabular interativo com *st.dataframe(hide_index=True)*. Presente em todos os visualizadores por herança.

3.3.2 Classes Especializadas

O projeto contém seis classes que herdam de Graficos, cada uma correspondendo a uma etapa ou perspectiva do fluxo de recebimento da BPJD. Todas seguem o mesmo contrato: sobrescrevem renderizar_metricas(), que calcula e exibe os valores das métricas e indicadores de desempenho, e renderizar_graficos(), que orquestra a sequência de renderização completa da aba.

GraficosVisaoGeral

Apresenta um panorama completo do fluxo. Define dois atributos de classe: COLUNAS_FLUXO, com as seis colunas de data que representam as etapas do processo, e LEGENDAS_EXIBICAO, com os rótulos legíveis correspondentes. Esses atributos são passados diretamente para grafico_funil() e grafico_linhas(), centralizando as definições do fluxo. renderizar_metricas() calcula: volume total recebido no período, taxa de escoamento (percentual com data_revisao_final preenchida) e tempo médio de ciclo (são os dias entre data_doacao e data_revisao_final). renderizar_graficos() encadeia: filtro por data_doacao → métricas → gráfico de barras horizontais por tipo_material → gráfico de funil de conversão → gráfico de linhas mensais.

GraficosDoacao

Analisa o fluxo de recebimento e triagem de materiais doados. renderizar_metricas() calcula: volume recebido, itens incorporados (incorporacao_acervo == True), itens recusados e taxa percentual de incorporação. renderizar_graficos() encadeia: filtro por data_doacao → métricas → gráfico de barras por doador (com hover de composição por material) → barras agrupadas comparando doações totais vs. incorporadas por tipo de material.

GraficosTombamento

Analisa o processo de tombamento patrimonial. renderizar_metricas() executa dois contextos independentes: o contexto de fluxo, que filtra por data_tombamento e calcula o total tombado e o tempo médio de resposta (dias entre data_doacao e data_tombamento); e o contexto de *backlog*, que expande a janela até a data mínima de doação, identifica os materiais incorporados sem data de tombamento no período. renderizar_graficos() encadeia: métricas → gráfico de pizza por tipo_material → gráfico de barras por responsável_tombamento → gráfico de barras cruzadas responsável × material → gráfico sunburst responsável por material.

GraficosCatalogacao

Analisa a produtividade de catalogação. renderizar_metricas() executa dois contextos: fluxo (total catalogado no período, dias ativos e produtividade diária) e *backlog* (materiais tombados até o final do período sem catalogação concluída, com janela expandida até a data mínima de tombamento). renderizar_graficos() encadeia: métricas → gráfico de pizza por tipo_material → gráfico de barras horizontais por tipo_material → gráfico sunburst responsável por material.

GraficosEstanteNovidades

Analisa as sub-etapas de digitalização, upload no SIGAA e liberação para a Estante de Novidades. renderizar_metricas() executa três contextos com filtros de data completamente independentes: digitalização (total digitalizado e pendentes de upload), upload

(total de uploads realizados) e estante (materiais com estante_novidades == True). renderizar_graficos() apresenta três seções com barras horizontais por tipo_material, uma para cada sub-etapa, cada uma com reset explícito de self.df antes do novo filtro.

GraficosPreparoFinal

Analisa a etapa física final: etiquetagem e inserção de papeleta. renderizar_metricas() executa dois contextos: fluxo (total com revisão final concluída, dias ativos e ritmo de acabamento) e backlog (materiais catalogados sem revisão final até o final do período, com janela expandida até a data mínima de catalogação). renderizar_graficos() encadeia métricas → gráfico de barras agrupadas comparando as três sub-etapas (Etiqueta Impressa, Etiqueta Colada, Papeleta Inserida) por tipo_material.

Algoritmo de Cálculo de Backlog Histórico

Todas as classes que calculam *backlog* (GraficosTombamento, GraficosCatalogacao e GraficosPreparoFinal) utilizam o mesmo algoritmo de janela expandida: o valor de self.data_inicial é substituído temporariamente pela data mínima da etapa antecedente registrada no banco, o *backlog* é calculado e o estado original é restaurado antes da renderização dos gráficos do período. Esse cálculo ocorre dentro do método renderizar_metrica().

```python
class GraficosTombamento(Graficos):
    def renderizar_metricas(self):
        # CONTEXTO 2: Calcular o Backlog Histórico
        # 1. Reseta o self.df para a planilha cheia para isolar o cálculo da fila
        self.df = self.data_manager.df.copy()

        # 2. Descobre dinamicamente a primeira data de registro de doações na biblioteca
        datas_doacao = self.df['data_doacao'].dropna()
        data_minima_real = datas_doacao.min() if not datas_doacao.empty else self.data_inicial

        # 3. Altera temporariamente a janela para capturar a fila desde o início do banco até data_final
        backup_data_inicial = self.data_inicial
        self.data_inicial = data_minima_real

        # 4. Aplica o filtro na coluna de entrada do processo (data_doacao)
        self.aplicar_filtro_por_coluna(coluna_data='data_doacao')

        # 5. Filtra a base para identificar materiais que já foram incorporados ao acervo mas ainda não receberam tombamento
        if self.df is not None and not self.df.empty:
            foi_incorporado = self.df['incorporacao_acervo'] == True
            nao_tombado_no_periodo = (self.df['data_tombamento'] > self.data_final) | (self.df['data_tombamento'].isna())
            backlog_tombamento = len(self.df[foi_incorporado & nao_tombado_no_periodo])
        else:
            backlog_tombamento = 0

        # 6. Restaura o estado original do filtro da tela
        self.data_inicial = backup_data_inicial
```

Figura 3.7: Trecho do algoritomo de backlog empregado no método renderizar_metricas() da classe GraficosTombamento.

3.4 Camada de Apresentação

A camada de apresentação é composta por três arquivos, app.py, inicio.py e estatisticas.py.

3.4.1 Ponto de Entrada (app.py)

O arquivo app.py configura a aplicação com *st.set_page_config()*, definindo título, ícone e layout da página, e centraliza a inicialização do *st.session_state* com as chaves data_inicial, data_final, categoria_selecionada e db_manager, que mantém as variavéis de filtro salvas para que elas não sejam reinicializadas durante a troca de página, melhorando consideravelmente a experiência do usuário. A instância de DadosBiblioteca é criada apenas quando a chave "db_manager"ainda não existe na sessão, garantindo que a conexão com a API seja estabelecida uma única vez por sessão de usuário. O roteamento entre páginas é gerenciado pelo *st.navigation()*, que registra os arquivos de cada página e constrói automaticamente o menu lateral de navegação. Cada página é definida como um objeto st.Page apontando para seu respectivo arquivo.

```python
st.set_page_config(
    page_title="Estatísticas BPJD",
    page_icon="📚",
    layout="wide"
)

# Força a criação das chaves de data ANTES de qualquer renderização
if "data_inicial" not in st.session_state:
    ano_atual = datetime.date.today().year
    st.session_state["data_inicial"] = datetime.date(ano_atual, 1, 1)

if "data_final" not in st.session_state:
    st.session_state["data_final"] = datetime.date.today()

if "categoria_selecionada" not in st.session_state:
    st.session_state["categoria_selecionada"] = "Visão Geral"

# Garante que o db_manager só seja instanciado uma vez na memória do servidor
if "db_manager" not in st.session_state:
    with st.spinner("Conectando à API do Google Sheets..."):
        st.session_state["db_manager"] = DadosBiblioteca('credenciais.json', "Fluxo de recebimento", 'Histórico completo')
        st.session_state["db_manager"].buscar_dados()

# Declaração das páginas apontando para os novos caminhos controlados
pagina_inicio = st.Page(
    page="src/screens/inicio.py",
    title="Início",
    icon="🏠",
    default=True
)

pagina_estatisticas = st.Page(
    page="src/screens/estatisticas.py",
    title="Estatísticas do Fluxo",
    icon="📊"
)

# Monta o menu de navegação
navegacao = st.navigation(
    {
        "": [pagina_inicio],
        "Análises": [pagina_estatisticas]
    }
)

navegacao.run()
```

Figura 3.8: *Script* completo do arquivo app.py.

Essa abordagem isola o código de cada página em seu próprio arquivo, evitando que o crescimento do número de páginas aumente a complexidade do ponto de entrada. O arquivo app.py permanece restrito à configuração global da aplicação, sem lógica de negócio ou de apresentação.

3.4.2 Página Inicial (inicio.py)

Página de boas-vindas. Exibe o título do sistema, os logotipos da BPJD e da EMUFRN lado a lado em colunas st.columns e uma descrição do projeto em Markdown.

3.4.3 Painel Principal (estatistica.py)

Página central do painel. Recupera o gerenciador de dados da sessão e exibe os filtros de data e categoria na barra lateral com a utilização do st.sidebar, para criação da barra lateral, *st.date_input* para filtros de data, e st.selectbox para a seleção das categorias de visualização. Os filtros de data expostos na barra lateral do Streamlit são submetidos a validações de integridade antes de serem propagados para os visualizadores. Duas condições de erro são verificadas: a data inicial ser maior que a data final, e a data inicial ser anterior à data mais antiga de doação registrada no banco de dados:

```
# Validação de erro para datas
erro_detectado = False

if dt_inicio_input > dt_final_input:
    st.sidebar.error("⚠️ A data inicial não pode ser maior que a data final.")
    erro_detectado = True

elif dt_inicio_input < menor_data_planilha:
    st.sidebar.error(f"⚠️ A data inicial não pode ser menor que o marco zero da planilha ({menor_data_planilha.strftime('%d/%m/%Y')}).")
    erro_detectado = True

# Se passar em todas as validações, atualiza a sessão global com segurança
if not erro_detectado:
    st.session_state["data_inicial"] = dt_inicio_input
    st.session_state["data_final"] = dt_final_input

    d_inicial = st.session_state["data_inicial"]
    d_final = st.session_state["data_final"]
else:
    # Se houver erro, mantém o último estado válido rodando nos gráficos abaixo para não quebrar o app
    d_inicial = st.session_state["data_inicial"]
    d_final = st.session_state["data_final"]
categoria_selecionada = st.session_state["categoria_selecionada"] = categoria_input
```

Figura 3.9: Trecho do código de validação de datas.

Quando um erro é detectado, o sistema não atualiza as chaves de estado da sessão e mantém o último estado válido como parâmetro dos visualizadores, evitando falhas de execução causadas por intervalos de data inválidos.

A seleção do visualizador ativo com base na categoria escolhida pelo usuário é implementada na página de estatísticas por meio de um dicionário que mapeia strings para classes:

```python
# Mapeamento dinâmico das classes
mapeamento_classes = {
    "Visão Geral": graficos.GraficosVisaoGeral,
    "Doações": graficos.GraficosDoacoes,
    "Tombamento": graficos.GraficosTombamento,
    "Catalogação": graficos.GraficosCatalogacao,
    "Estante de Novidades": graficos.GraficosEstanteNovidades,
    "Preparo Final": graficos.GraficosPreparoFinal
}

# Instanciação dinâmica passando o dataframe via Construtor
if categoria_selecionada in mapeamento_classes:
    visualizador_ativo = mapeamento_classes[categoria_selecionada](db_manager, d_inicial, d_final)
    visualizador_ativo.renderizar_graficos()
```

Figura 3.10: Trecho do código do mapeamento dinâmico das classe através de um dicionário.

Essa abordagem elimina cadeias de condicionais if/elif e aplica o princípio do polimorfismo: a página de estatísticas não precisa conhecer os detalhes de nenhuma classe específica, apenas invocar renderizar_graficos() na instância retornada. Adicionar uma nova categoria de análise no futuro requer apenas a criação de uma nova subclasse e uma nova entrada no dicionário. A seguir o diagrama de sequência que ilustra todo o fluxo de execução da aplicação:

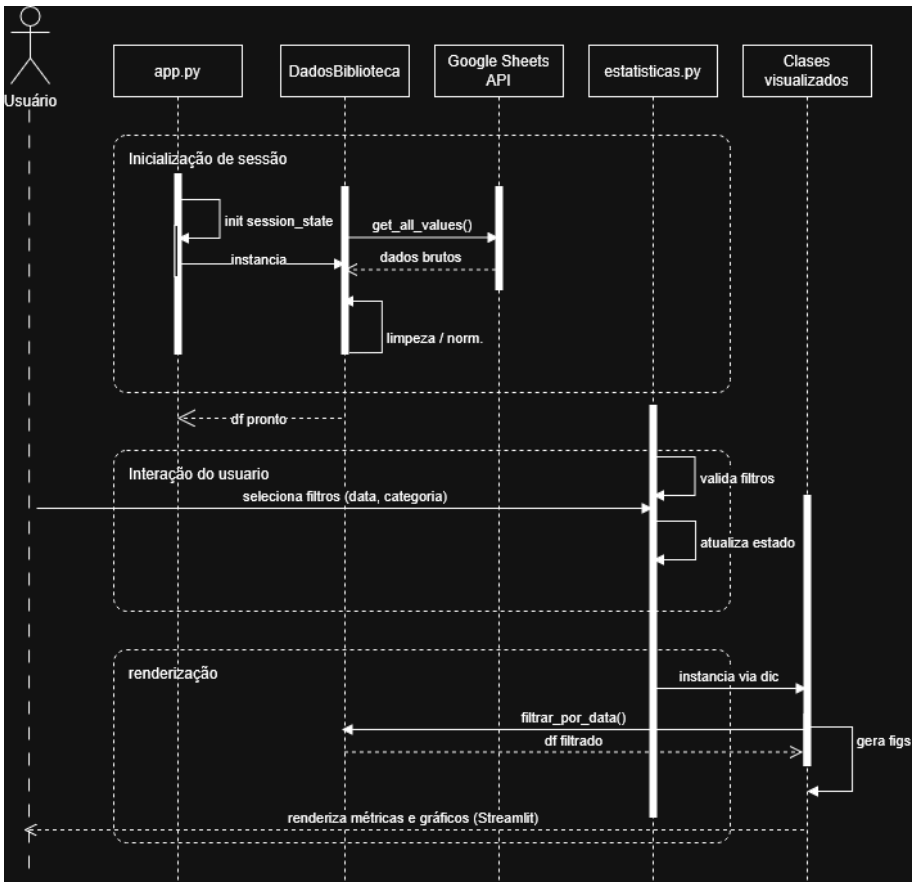


Figura 3.11: Diagrama UML de sequência.

3.5 Interface e Especificação Funcional dos Módulos do Painel

A partir do mapeamento dinâmico executado pelo script estatisticas.py (conforme detalhado na seção 3.4.3), o sistema instancia a classe correspondente à categoria selecionada na barra lateral. Cada um desses módulos foi projetado com regras de negócio específicas para atender às demandas de gerenciamento da BPJD, combinando um painel superior de indicadores de desempenho e um conjunto inferior de gráficos interativos. A seguir, especificam-se o escopo e a composição de cada módulo disponível no sistema.

3.5.1 Módulo: Visão Geral

No módulo de "Visão Geral", o sistema reúne os principais dados e gráficos do projeto. Bem no topo da tela, o usuário visualiza um painel de indicadores que mudam de acordo com o período de tempo selecionado. Esse painel destaca três informações principais:

- Total de Materiais Recebidos: Mostra a quantidade exata de itens que entraram na biblioteca como doação.
- Taxa de Eficiência: É o percentual de materiais que conseguiram passar por todo o fluxo de trabalho com sucesso, ou seja, aqueles que já tiveram a revisão final concluída.
- Tempo Médio de Ciclo: Calcula quantos dias, em média, um material leva desde o momento em que chega na biblioteca até ser revisado e finalizado.

Logo abaixo desse painel de indicadores, o dashboard exibe três gráficos interativos para ajudar a explorar os dados:

- Gráfico de Barras Horizontais: Mostra a quantidade de materiais dividida por tipo (como livros, partituras, CDs, etc.), facilitando ver o que mais chega na biblioteca.
- Gráfico de Funil: Serve para acompanhar o andamento do fluxo de trabalho. Com ele, dá para ver facilmente quantos materiais estão em cada etapa e descobrir em qual fase o processo está travando (os chamados gargalos).
- Gráfico de Linhas: Mostra a evolução das etapas ao longo dos meses, o que ajuda o gestor a entender o ritmo de trabalho e a produtividade da equipe com o passar do tempo.

No final de cada painel é possível exibir a tabela geral com os registros filtrados pela coluna de data que corresponde a sua categoria.

3.5.2 Módulo: Doações

No módulo de "Doações", o painel de indicadores no topo da tela foi configurado com quatro métricas específicas para avaliar a qualidade e a triagem do que chega à instituição.

- Total de Materiais Recebidos: Exibe o volume bruto de itens que deram entrada como doação (equivalente ao contador da "Visão Geral").

- Total de Materiais Incorporados ao Acervo: Contabiliza os itens que passaram com sucesso pela triagem técnica e foram selecionados para fazer parte do acervo permanente da biblioteca.
- Total de Materiais Rejeitados: Mostra o volume de itens que não passaram nos critérios da triagem e receberão outro destino (como doação para outras instituições ou descarte), variando de acordo com o estado de conservação do material.
- Taxa de Incorporação: Apresenta o percentual de materiais aproveitados em relação ao total recebido, servindo como um indicador de relevância das doações.

Abaixo dessas métricas, o módulo oferece duas visualizações gráficas focadas nos doadores e nos tipos de suporte:

- Gráfico de Barras por Doador: Apresenta o volume de doações trazido por cada pessoa ou entidade. Este gráfico conta com um hover customizado que, ao passar o mouse sobre a barra, revela a lista detalhada e a quantidade de cada tipo de material entregue por aquele doador específico.
- Gráfico de Barras Agrupadas: Coloca lado a lado, para cada tipo de material (livros, partituras, mídias), o total de itens recebidos em comparação com o total que foi efetivamente incorporado. Essa visualização ajuda a entender quais tipos de materiais são mais aproveitados pela equipe.

3.5.3 Módulo: Tombamento

No módulo de "Tombamento", o painel de indicadores foi configurado com três métricas específicas para avaliar o andamento e o desempenho desse processo de registro patrimonial:

- Total de Patrimônios Tombados: Exibe o volume bruto de materiais que receberam um número de tombamento dentro do período selecionado.
- Total de Materiais Aguardando Tombamento: Mostra o acúmulo de trabalho (*backlog*) de itens que foram aprovados na triagem para incorporação, mas que estão com a data de tombamento vazia ou programada para depois do período selecionado.
- Tempo Médio para Tombar: Calcula a média de dias que a equipe leva para realizar o tombamento do material, medindo o intervalo entre a data de chegada da doação e a data do registro do patrimônio.

Em seguida, o dashboard apresenta quatro gráficos interativos que cruzam os dados dos materiais tombados com os responsáveis pela execução do trabalho:

- Gráfico de Pizza por Tipo de Material: Exibe a proporção de itens tombados por categoria. Para evitar poluição visual, as categorias com 10 ou menos itens são agrupadas automaticamente em uma única fatia chamada "Outros", contando com um *hover* customizado que detalha quais itens estão escondidos ali quando o usuário passa o *mouse*.

- Gráfico de Barras por Operador: Mostra a quantidade absoluta de tombamentos realizados por cada funcionário ou bolsista, facilitando o acompanhamento da produtividade.
- Gráfico de Barras Cruzadas: Cruza as informações ao mostrar a quantidade de materiais tombados por cada responsável, mas dividindo as barras internamente por tipo de material.
- Gráfico de Barras Cruzadas: Cruza as informações ao mostrar a quantidade de materiais tombados por cada responsável, mas dividindo as barras internamente por tipo de material.
- Gráfico Sunburst (Hierárquico): Apresenta uma visualização circular em camadas que conecta os responsáveis pelo tombamento (no anel interno) com os tipos de materiais que cada um processou (no anel externo), permitindo uma exploração dinâmica da distribuição de tarefas.

3.5.4 Módulo: Catalogação

No módulo de "Catalogação", o sistema disponibiliza três indicadores fundamentais para monitorar o ritmo de inserção técnica das obras no sistema da biblioteca:

- Total de Obras Catalogadas: Exibe a quantidade exata de itens que receberam registro catalográfico dentro do período selecionado.
- Aguardando Catalogação: Apresenta o acúmulo de trabalho (*backlog*) desta etapa, contabilizando as obras que já foram tombadas, mas que estão com a data de catalogação vazia ou posterior ao intervalo selecionado.
- Produtividade Média Diária: Calcula a média de materiais que a equipe da biblioteca consegue catalogar por dia útil trabalhado, servindo como um indicador de eficiência operacional.

Logo abaixo, são renderizados três gráficos interativos para detalhar esses registros:

- Gráfico de Pizza por Tipo de Material: Mostra a proporção percentual de cada tipo de material catalogado em relação ao todo.
- Gráfico de Barras Horizontais por Tipo de Material: Complementa a visualização anterior, permitindo comparar de forma direta e em valores absolutos o volume de processamento de cada categoria (como livros, partituras e folhetos).
- Gráfico Sunburst (Hierárquico): Apresenta a relação entre os funcionários responsáveis pela catalogação (camada interna) e os tipos de materiais que cada um processou (camada externa), facilitando o entendimento de como as demandas foram distribuídas na equipe.

3.5.5 Módulo: Estante de Novidades

No módulo da "Estante de Novidades", o dashboard exibe quatro métricas essenciais para acompanhar as fases finais de digitalização e disponibilização pública dos materiais:

- Materiais Digitalizados: Mostra o volume total de itens que tiveram suas capas e sumários digitalizados pela equipe, estando prontos para a fase de inserção no sistema.
- Uploads Concluídos: Contabiliza a quantidade de materiais que já tiveram seus arquivos e metadados enviados com sucesso para o sistema integrado (SIGAA).
- Aguardando Upload: Apresenta o acúmulo de trabalho (*backlog*) desta etapa, indicando quantos itens já foram digitalizados, mas ainda aguardam o envio para o sistema.
- Disponíveis para a Estante: Exibe a quantidade de materiais que já passaram pela revisão final de catalogação e estão prontos e liberados para compor fisicamente ou virtualmente a estante de novidades da biblioteca.

Abaixo do painel de indicadores, o sistema renderiza três gráficos de barras horizontais. Cada um deles foi projetado para detalhar a quantidade total de materiais por tipo (como livros, partituras, etc.) em cada uma das subetapas que formam esse processo:

- Etapa de Digitalização: Detalha os tipos de materiais cujas capas e sumários já foram processados no scanner.
- Etapa de Upload: Mostra a distribuição por categoria dos itens que já foram integrados ao sistema institucional.
- Etapa de Liberação: Apresenta o perfil dos materiais que já completaram o fluxo e estão disponíveis na estante de novidades para consulta dos usuários.

3.5.6 Módulo: Preparo Final

No módulo de "Preparo Final", o dashboard apresenta três indicadores essenciais para monitorar o acabamento físico e a liberação definitiva dos itens para o acervo:

- Materiais Concluídos: Mostra a quantidade total de itens que finalizaram com sucesso todas as etapas do fluxo operacional, tendo a data de revisão final preenchida.
- Aguardando Preparo: Apresenta o acúmulo de trabalho (backlog) atual desta fase, contabilizando os materiais que já foram catalogados, mas que ainda não passaram pelos procedimentos de acabamento físico.
- Ritmo de Acabamento: Calcula a média diária de materiais preparados pela equipe dentro do período de tempo selecionado, servindo para acompanhar a velocidade de entrega do setor.

Logo abaixo, o módulo exibe um componente visual focado nos detalhes práticos dessa última fase:

- Gráfico de Barras Agrupadas por Tipo de Material: Coloca lado a lado o volume de itens processados em cada uma das três subetapas do preparo físico: a impressão de etiquetas, a colagem de etiquetas e a colagem de papeletas de empréstimo. Essa visualização permite ao gestor identificar se há algum atraso específico em uma dessas pequenas tarefas manuais, segmentado por tipo de material.

Capítulo 4

Experimentos e Resultados

A implementação do sistema proposto para a Biblioteca Setorial da Escola de Música Padre Jaime Diniz (BPJD), pertencente à Universidade Federal do Rio Grande do Norte (UFRN), foi concluída com êxito, cumprindo os requisitos levantados junto à equipe técnica da instituição. O sistema viabilizou uma análise quantitativa e visual, de forma clara e intuitiva, dos processos que integram o fluxo operacional — desde o recebimento de novos materiais por meio de doações até a sua efetiva inserção no acervo físico. A extração dos dados ocorre em tempo real a partir das informações persistidas no Google Sheets, ferramenta utilizada no registro cotidiano por bolsistas e funcionários. A interface web desenvolvida com o Streamlit permite que o gestor filtre categorias e intervalos temporais específicos para exploração, revelando padrões e métricas que não eram facilmente identificáveis por meio da visualização isolada de tabelas estáticas.

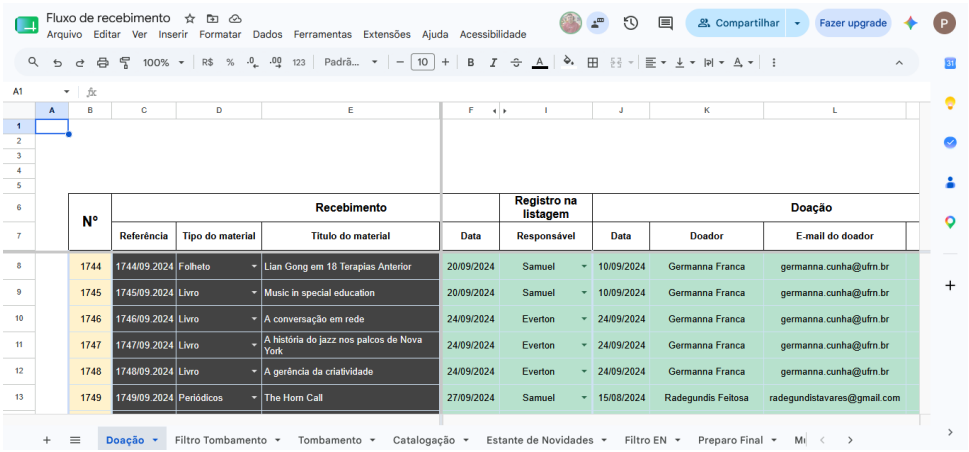
O sistema demonstrou eficiência ao consolidar dados quantitativos obtidos por meio da manipulação de dados com a biblioteca Pandas, gerando métricas de volume de trabalho e indicadores de desempenho, tais como o acúmulo de tarefas (backlog) histórico entre as etapas de processamento. A integração do Plotly como motor de renderização gráfica conferiu interatividade às análises, permitindo a compreensão visual imediata do fluxo de recebimento. Adicionalmente, a arquitetura baseada no gerenciamento de estado global via st.session_state garantiu uma experiência de uso fluida, preservando o estado dos filtros aplicados e das configurações customizadas durante a navegação entre as páginas do painel.

Resumindo, o projeto desenvolvido cumpriu o objetivo de otimizar a agregação e a análise dos dados gerados no fluxo de trabalho de recebimento e processamento técnico dos materiais doados à BPJD, entregando uma plataforma intuitiva, responsiva e orientada a dados.

4.1 Cenário Anterior à Implementação

Antes da introdução do painel analítico, o acompanhamento do fluxo de processamento técnico da BPJD era realizado inteiramente na planilha do Google Sheets, estruturada em múltiplas abas — uma para cada etapa do processo (Doação, Tombamento, Catalogação, Estante de Novidades, Preparo Final), conforme ilustrado na Figura 4.1. Cada registro era inserido manualmente pelos bolsistas e funcionários, sem validação au-

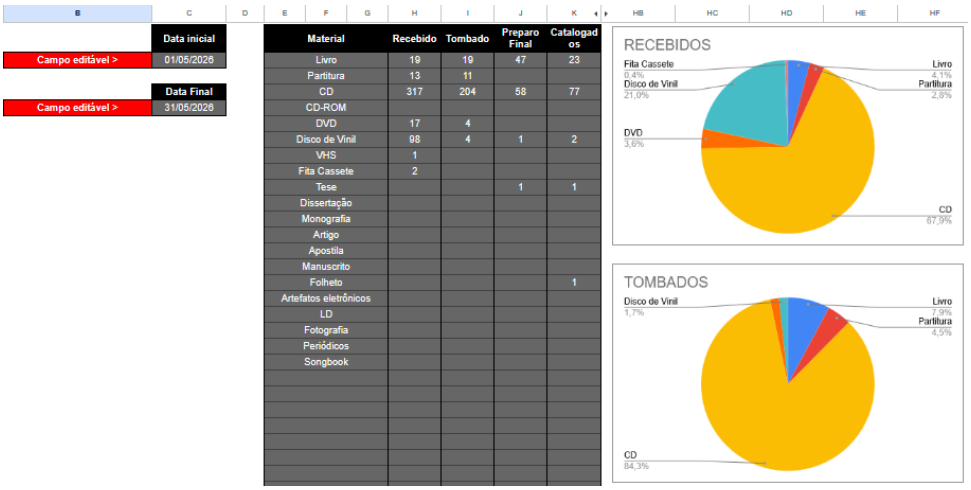
tomática de datas e sem qualquer mecanismo de filtragem dinâmica entre as etapas.



Nº	Recebimento				Registro na listagem	Doação		
	Referência	Tipo do material	Título do material	Data	Responsável	Data	Doador	E-mail do doador
1744	1744/09.2024	Folheto	Lian Gong em 18 Terapias Anterior	20/09/2024	Samuel	10/09/2024	Germanna Franca	germanna.cunha@ufrn.br
1745	1745/09.2024	Livro	Music in special education	20/09/2024	Samuel	10/09/2024	Germanna Franca	germanna.cunha@ufrn.br
1746	1746/09.2024	Livro	A conversação em rede	24/09/2024	Everton	24/09/2024	Germanna Franca	germanna.cunha@ufrn.br
1747	1747/09.2024	Livro	A história do jazz nos palcos de Nova York	24/09/2024	Everton	24/09/2024	Germanna Franca	germanna.cunha@ufrn.br
1748	1748/09.2024	Livro	A gerência da criatividade	24/09/2024	Everton	24/09/2024	Germanna Franca	germanna.cunha@ufrn.br
1749	1749/09.2024	Periódicos	The Horn Call	27/09/2024	Samuel	15/08/2024	Radegundis Feitosa	radegundistavares@gmail.com

Figura 4.1: Estrutura tabular da planilha de fluxo de recebimento, com registros inseridos manualmente por aba.

Para a consolidação dos relatórios mensais, a planilha contava com uma aba adicional, denominada "Estatística"(Figura 4.2), na qual os totais de materiais Recebidos, Tombados, Catalogados e com Preparo Final concluído eram calculados por meio da função nativa `SOMA`, aplicada sobre os registros compreendidos entre as datas informadas manualmente pelo operador nos campos "Data Inicial"e "Data Final". A partir desses totais, gráficos de pizza eram gerados para exibir a proporção de cada tipo de material recebido e tombado no período. Apesar de automatizar parcialmente a contagem, essa abordagem dependia inteiramente da atualização manual do intervalo de datas para refletir um novo período de análise, e não calculava indicadores mais elaborados — como o backlog acumulado entre etapas, o tempo médio de processamento ou a produtividade por responsável.



Material	Recebido	Tombado	Preparo Final	Catalogados
Livro	19	19	47	23
Partitura	13	11		
CD	317	204	58	77
CD-ROM				
DVD	17	4		
Disco de Vinil	98	4	1	2
VHS	1			
Fita Cassete	2			
Tese			1	1
Dissertação				
Monografia				
Artigo				
Apostila				
Manuscrito				
Folheto				1
Artefatos eletrônicos				
LD				
Fotografia				
Periódicos				
Songbook				

Figura 4.2: Aba "Estatística"da planilha original, com filtros de data editados manualmente e totais calculados pela função `SOMA`.

Esse cenário evidencia as limitações discutidas no Capítulo 1: a dependência de atualização manual das fórmulas e do intervalo de datas, a ausência de indicadores de backlog e produtividade, e a necessidade de intervenção humana para qualquer nova análise. É justamente esse conjunto de deficiências que o sistema desenvolvido neste trabalho se propõe a resolver, como demonstrado a seguir.

4.2 Sistema Implementado

A seguir serão apresentados os exemplos dos principais gráficos e informações geradas com o sistema em execução. As primeiras interfaces apresentadas correspondem à tela inicial e à estrutura de navegação do sistema, representadas pelas Figuras 4.3 e 4.4.



Figura 4.3: Tela inicial do sistema.

A figura 4.3 exibe a tela de boas-vindas da aplicação (inicio.py), projetada com uma descrição sucinta do projeto e alinhada à identidade visual institucional da BPJD e da EMUFRN.

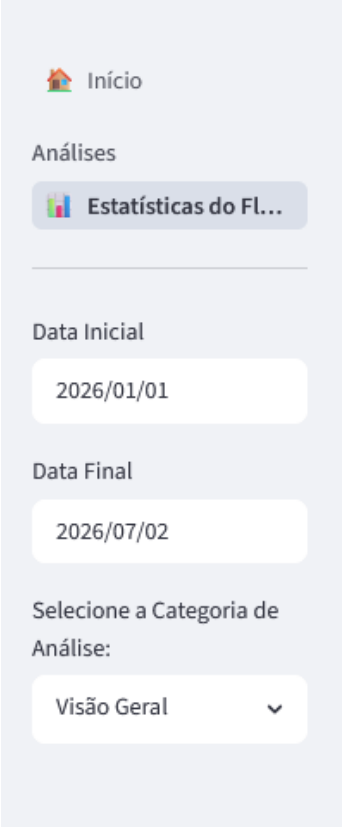


Figura 4.4: Barra lateral com navegação de páginas e seletores de filtros de data e categoria.

A figura 4.4 destaca o menu lateral gerado pelo componente de navegação do Streamlit. Na prática, essa interface confirma a facilidade de uso do sistema, organizando as etapas do fluxo técnico de forma limpa e permitindo que o operador transite entre os módulos mantendo o estado dos seus filtros salvos na barra lateral durante toda a sessão de uso.

A seguir será mostrado os módulos analíticos preenchidos com os dados reais da biblioteca.

Ao selecionar a página de estatísticas, o menu lateral revela os seletores de filtros, configurados por padrão para abranger o intervalo do primeiro dia do ano corrente até a data atual, e renderiza a interface principal, o módulo de "Visão Geral", que está representada na figura 4.5.



Figura 4.5: Painel do Módulo de Visão Geral.

Na imagem, observa-se carregamento imediato dos cartões de métricas e dos três gráficos (barras, funil e linhas), que são visualizados navegando pela página com o scroll do mouse, confirma o processamento síncrono dos dados e exibe o panorama geral do fluxo da biblioteca. A dinâmica de conversão volumétrica de todo o ciclo operacional da biblioteca pode ser acompanhada de forma centralizada por meio do gráfico de funil, detalhado na figura 4.6.

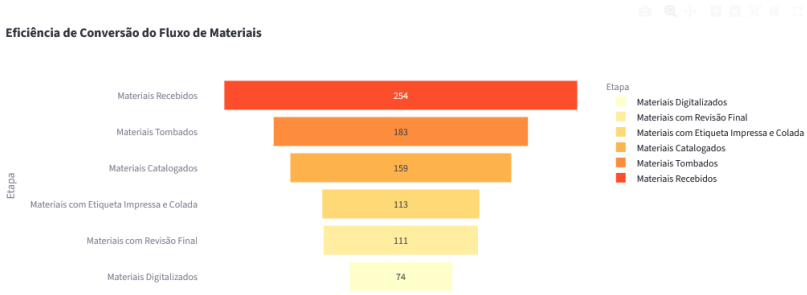
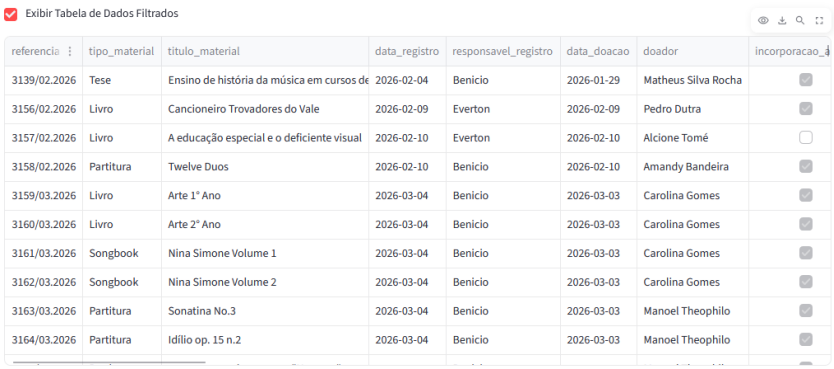


Figura 4.6: Gráfico de funil das etapas do fluxo de processamento.

O gráfico de funil comprova visualmente a capacidade do sistema em integrar dados de diferentes colunas temporais de maneira síncrona. A partir do volume inicial de 252 materiais recebidos, o gestor consegue rastrear o escoamento quantitativo entre as etapas subsequentes (183 tombados, 159 catalogados, até os processos de acabamento). Essa visualização é crucial para que a chefia da biblioteca identifique, de forma imediata, em qual fase do processamento técnico se encontra o maior volume de pendências, otimizando o planejamento de trabalho.

No final de cada painel é possível exibir a tabela geral com os registros filtrados pela coluna de data que corresponde a sua categoria:



referencia	tipo_material	titulo_material	data_registro	responsavel_registro	data_doacao	doador	incorporacao_a
3139/02.2026	Tese	Ensino de história da música em cursos de	2026-02-04	Benicio	2026-01-29	Matheus Silva Rocha	☑
3156/02.2026	Livro	Cancioneiro Trovadores do Vale	2026-02-09	Everton	2026-02-09	Pedro Dutra	☑
3157/02.2026	Livro	A educação especial e o deficiente visual	2026-02-10	Everton	2026-02-10	Alcione Tomé	☐
3158/02.2026	Partitura	Twelve Duos	2026-02-10	Benicio	2026-02-10	Amandy Bandeira	☑
3159/03.2026	Livro	Arte 1° Ano	2026-03-04	Benicio	2026-03-03	Carolina Gomes	☑
3160/03.2026	Livro	Arte 2° Ano	2026-03-04	Benicio	2026-03-03	Carolina Gomes	☑
3161/03.2026	Songbook	Nina Simone Volume 1	2026-03-04	Benicio	2026-03-03	Carolina Gomes	☑
3162/03.2026	Songbook	Nina Simone Volume 2	2026-03-04	Benicio	2026-03-03	Carolina Gomes	☑
3163/03.2026	Partitura	Sonatina No.3	2026-03-04	Benicio	2026-03-03	Manoel Theophilo	☑
3164/03.2026	Partitura	Idílio op. 15 n.2	2026-03-04	Benicio	2026-03-03	Manoel Theophilo	☑

Figura 4.7: Tabela de registros.

Além de consolidar os dados quando os parâmetros estão corretos, a robustez da camada de controle do sistema foi testada por meio de rotinas de validação de dados na barra lateral. O sistema foi programado para interceptar consultas inconsistentes antes que elas disparem requisições inválidas à API ou quebrem a renderização dos gráficos. Esse algoritmo de validação foi apresentado na figura 3.9 do capítulo anterior. A imagem 4.8 ilustra o comportamento da interface quando uma incongruência cronológica é detectada.

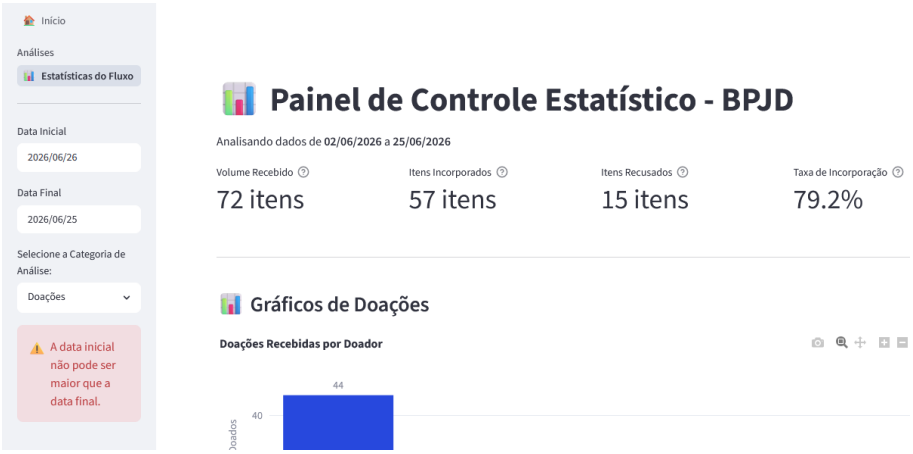


Figura 4.8: Mensagem de erro exibida na validação do filtro de datas.

Como observado na imagem, ao tentar configurar uma data inicial posterior à data final, o sistema bloqueia a execução, mantendo o estado das datas anteriores para não quebrar os gráficos, e renderiza um alerta *st.sidebar.error*, orientando o usuário a corrigir o intervalo. Adicionalmente, a mesma lógica de proteção foi validada para um segundo cenário crítico: caso o operador selecione uma data inicial retroativa ao "marco zero"da planilha (limite inferior do histórico de registros da BPJD), o painel emite um aviso correspondente e impede o processamento de valores inválidos. Essa validação de restrições garante a estabilidade da aplicação em ambiente de produção, tratando exceções de entrada diretamente na interface de usuário.

Avançando para a triagem inicial do fluxo, o módulo de doações permite mapear o perfil das contribuições recebidas pela instituição, conforme ilustrado na figura 4.9. Todos os gráficos a seguir foram gerados com o filtro temporal correspondente ao mês de junho (01/06/2026 a 30/06/2026).

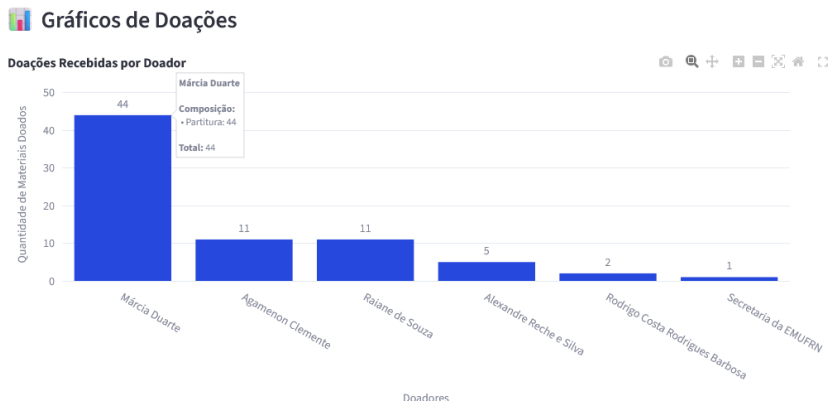


Figura 4.9: Gráfico de Barras da quantidade de doações recebidas por doador.

Este gráfico valida o requisito de interatividade da camada de visualização: ao posicionar o cursor sobre a barra do doador com maior volume (Márcia Duarte, com 44 materiais), o sistema renderiza em tempo real um balão de informações (*hover*) customizado que detalha a composição interna daquele montante, composto integralmente por partituras. Essa quebra dinâmica evita a necessidade de gerar múltiplos gráficos secundários para entender o perfil de cada doação.

No processamento patrimonial do acervo, a distribuição de registros por tipo de item é exibida por meio do gráfico de pizza em formato de rosquinha, visível na figura 4.10.

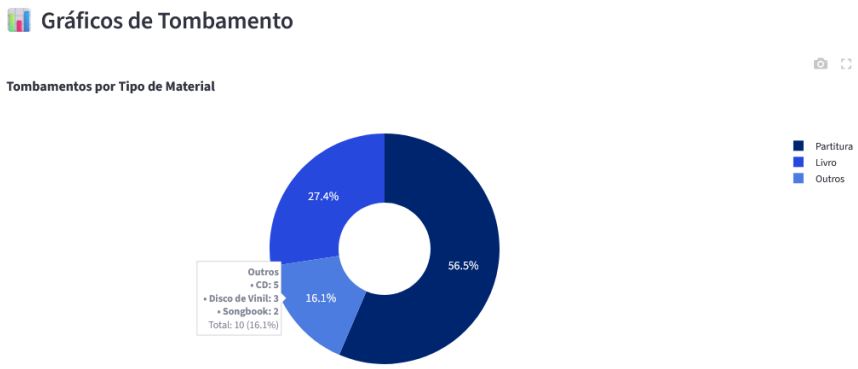


Figura 4.10: Gráfico de Pizza de quantidade de tombamentos por tipo de material.

Essa imagem valida com sucesso o algoritmo implementado para o tratamento de categorias minoritárias, uma vez que itens com volume menor ou igual a 10 (como CD,

Disco de Vinil e Songbook) foram condensados automaticamente em uma única fatia denominada "Outros", que responde por 16,1% do total. O balão de informações detalha os valores isolados de cada categoria oculta ao passar o mouse, mantendo o gráfico limpo e legível sem perder a precisão dos dados. A produtividade da equipe e a divisão de atribuições na catalogação são analisadas de forma multinível através do gráfico apresentado na figura 4.11.

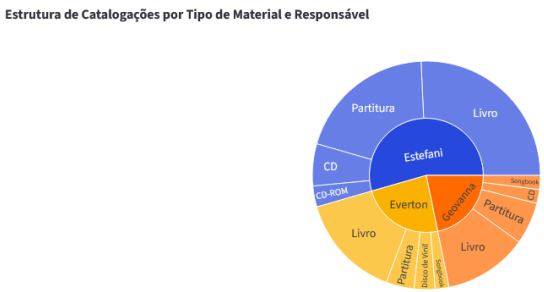


Figura 4.11: Gráfico Sunburst de catalogação por responsável e tipo de material.

O gráfico Sunburst gerado prova a capacidade do sistema de cruzar e estruturar dados multidimensionais sem sobrepor informações na tela. O anel interno mapeia os operadores responsáveis (como Estefani, Everton e Geovanna), enquanto o anel externo segmenta os tipos de materiais processados por cada um deles, fornecendo ao gestor um diagnóstico imediato sobre a especialização ou sobrecarga de tarefas na equipe. Por fim, o encerramento do fluxo técnico é monitorado pelas ferramentas do módulo de preparo final. O painel superior de indicadores dessa tela, exibido na figura 4.12, valida o algoritmo de *backlog* desenvolvido na pesquisa.



Figura 4.12: Painel de indicadores da categoria Preparo Final.

Conforme observado no painel, o sistema contabiliza com precisão 75 itens concluídos e prontos para irem à estante, enquanto calcula um ritmo médio de 18.8 itens preparados por dia trabalhado. A eficácia do algoritmo de varredura histórica se consolida no indicador central de *backlog*: ao expandir a janela temporal de forma independente, o painel expõe um passivo real de 167 itens aguardando preparo, sinalizando visualmente um gargalo técnico que exige atenção gerencial.

Logo abaixo, esse monitoramento é complementado pelo gráfico de barras agrupadas, visível na figura 4.13, que coloca lado a lado os volumes correspondentes às três micro-tarefas manuais de finalização (Impressão de Etiquetas, Colagem de Etiquetas e Inserção de Papeleta). Embora cada subetapa corresponda a uma coluna de data distinta e independente na planilha original, o alinhamento perfeito das barras em valores idênticos para cada categoria (como os 53 livros e 16 CDs) serve como uma rigorosa validação lógica do sistema. Esse resultado reflete com fidelidade a dinâmica operacional da BPJD, onde o processamento físico dos materiais de um mesmo lote costuma ser executado e concluído de forma sequencial no mesmo dia.

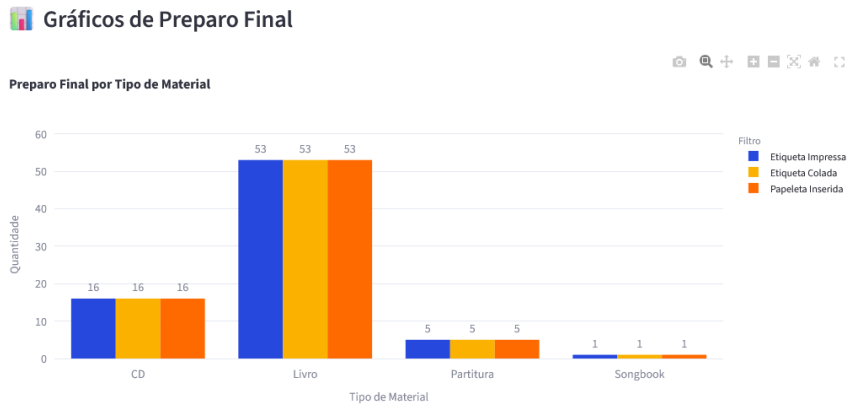


Figura 4.13: Gráfico de Barras Agrupadas da categoria Preparo Final.

Capítulo 5

Conclusão

Este trabalho apresentou o desenvolvimento e a implementação de um sistema interativo para monitoramento e análise do fluxo de processamento técnico de materiais doados na Biblioteca Padre Jaime Diniz (BPJD/UFRN), utilizando as tecnologias Python, Streamlit, Plotly, Pandas e Gspread. A solução foi concebida com o propósito de superar as limitações do acompanhamento realizado exclusivamente por meio de planilhas eletrônicas, oferecendo uma plataforma capaz de consolidar, processar e visualizar os dados de maneira dinâmica e intuitiva.

Os resultados obtidos demonstram que o sistema atingiu os objetivos propostos, possibilitando a extração automática dos dados armazenados no Google Sheets, sua organização em uma arquitetura orientada a objetos e a disponibilização de indicadores gerenciais por meio de dashboards interativos. Os recursos implementados permitiram identificar gargalos no fluxo de processamento, acompanhar a produtividade dos responsáveis por cada etapa, calcular o backlog histórico e gerar relatórios de forma significativamente mais rápida e precisa, contribuindo para uma gestão baseada em dados.

Além dos ganhos operacionais, a arquitetura modular adotada favorece a manutenção e a evolução do sistema, permitindo a incorporação de novas funcionalidades sem alterações significativas na estrutura principal da aplicação. A utilização do Streamlit também demonstrou ser uma alternativa eficiente para o desenvolvimento de aplicações analíticas, proporcionando uma interface responsiva, fluida e de fácil utilização para a coordenação da biblioteca.

Dessa forma, conclui-se que a ferramenta desenvolvida representa uma contribuição prática para a gestão da BPJD, apoiando a tomada de decisões e tornando mais eficiente o acompanhamento do processamento técnico dos materiais incorporados ao acervo. O projeto também evidencia o potencial da aplicação de técnicas de Engenharia de Software e Ciência de Dados na modernização de processos administrativos, servindo como modelo viável para outras bibliotecas universitárias e demais instituições públicas.

Referências Bibliográficas

BIBLIOTECA PADRE JAIME DINIZ (2026), 'Biblioteca padre jaime diniz', https://biblioteca.musica.ufrn.br/?page_id=43. Acesso em: 4 jul. 2026.

DRAW.IO (2026), 'Uml diagrams', https://www.drawio.com/docs/diagram-types/uml/. Acesso em: 4 jul. 2026.

GONÇALVES, NATHÁLIA (2018), 'Gspread: trabalhando com o google sheets', https://medium.com/pyladiesbh/gspread-trabalhando-com-o-google-sheets-f12e53ed1346. Acesso em: 4 jul. 2026.

GOOGLE CLOUD (2026), 'Google cloud documentation', https://docs.cloud.google.com/docs?authuser=1&hl=pt-br. Acesso em: 4 jul. 2026.

PANDAS DEVELOPMENT TEAM (2026), 'pandas documentation', https://pandas.pydata.org/docs/. Acesso em: 4 jul. 2026.

PLOTLY (2026), 'Python graphing library', https://plotly.com/python/. Acesso em: 4 jul. 2026.

PYTHON SOFTWARE FOUNDATION (2026*a*), 'Classes — the python tutorial', https://docs.python.org/3/tutorial/classes.html. Acesso em: 4 jul. 2026.

PYTHON SOFTWARE FOUNDATION (2026*b*), 'datetime — basic date and time types', https://docs.python.org/3/library/datetime.html. Acesso em: 4 jul. 2026.

PYTHON SOFTWARE FOUNDATION (2026*c*), 'Python', https://www.python.org/. Acesso em: 4 jul. 2026.

STREAMLIT (2026*a*), 'Streamlit', https://streamlit.io/. Acesso em: 4 jul. 2026.

STREAMLIT (2026*b*), 'Streamlit documentation', https://docs.streamlit.io/. Acesso em: 4 jul. 2026.